Resumo – OO: Modelagem e Relacionamento

1. O ponto de partida

Na aula anterior vimos classe e objeto, atributos e métodos, `self` e `__init__`. Uma classe organiza dados e comportamentos relacionados. Mas, em um sistema de verdade, os objetos **não ficam sozinhos**: um personagem coleta um item, tem uma arma, uma fase tem vários inimigos, um inimigo ataca um personagem.

A pergunta desta aula é: **como modelar objetos que se relacionam?**

2. Modelagem: como identificar as classes

Diante de um enunciado, pergunte:

- quais elementos são importantes e aparecem várias vezes?
- quais possuem dados próprios?
- quais executam ações?
- quais precisam ser manipulados no código?

Macete

Substantivos viram classes (ou atributos); verbos viram métodos. Sublinhe os substantivos do enunciado. Se o substantivo tem dados próprios e comportamento, é uma **classe**. Se é só uma informação simples sobre outra coisa (um nome, um valor), é um **atributo**.

Exemplo da aula

"O personagem percorre fases. Cada fase tem inimigos e itens espalhados pelo mapa. O personagem pode se mover, coletar itens e perder vida. Os inimigos atacam e se movem."

Personagem	**Inimigo**	**Item**	**Fase**
nome vida x, y	nome vida x, y	nome valor x, y	nome inimigos itens

`mover()`
`perder_vida()`
`coletar()`
	`atacar()`
	`mover()`
		`exibir()`	`carregar()`
			`exibir_elementos()`

Repare na classe `Fase`: os atributos `inimigos` e `itens` não são números nem textos, são **outros objetos**. Um objeto pode guardar outros objetos. Essa é a porta de entrada para as três relações.

3. As três relações

Usa	Tem um	Tem vários
recebe o objeto como **parâmetro**	guarda o objeto em um **atributo**	guarda uma **lista** de objetos em um atributo
`heroi.coletar(moeda)`	`self.arma = arma`	`self.inimigos = []`
Personagem usa Item	Personagem tem uma Arma	Fase tem vários Inimigos
dura só durante o método	dura enquanto o objeto existir	dura enquanto o objeto existir

Macete

Para descobrir a relação, leia a frase em voz alta. "O personagem **coleta** um item": é uma ação momentânea, então **usa** (parâmetro). "O personagem **tem uma** arma": ele carrega a arma o tempo todo, então **atributo**. "A fase **tem vários** inimigos": coleção, então **lista como atributo**.

4. Relação "usa"

Um método pode receber outro objeto como parâmetro e acessar os dados dele.

```python
class Item:
    def __init__(self, nome, valor):
        self.nome = nome
        self.valor = valor


class Personagem:
    def __init__(self, nome, pontos):
        self.nome = nome
        self.pontos = pontos

    def coletar(self, item):
        self.pontos += item.valor
        print(f"{self.nome} coletou {item.nome}")


heroi = Personagem("Heroi", 0)
moeda = Item("Moeda", 10)

heroi.coletar(moeda)
print(heroi.pontos)
```

```
Heroi coletou Moeda
10
```

Dentro de `coletar`, o parâmetro `item` e a variável `moeda` são o **mesmo objeto**. O método acessa `item.valor` e `item.nome`: dados de outro objeto. Depois que o método termina, o personagem não guarda a moeda.

Atenção

Dentro do método há dois objetos: `self` (o personagem) e `item` (a moeda). Cuidado para não trocar: `self.nome` é "Heroi" e `item.nome` é "Moeda".

5. Relação "tem um"

Um objeto pode guardar outro objeto como atributo. O personagem tem uma arma e pede para a arma agir.

```
class Arma:
    def __init__(self, nome, dano):
        self.nome = nome
        self.dano = dano

    def usar(self):
        print(f"{self.nome} causa {self.dano} de dano")


class Personagem:
    def __init__(self, nome, arma):
        self.nome = nome
        self.arma = arma

    def atacar(self):
        print(f"{self.nome} ataca!")
        self.arma.usar()


espada = Arma("Espada", 8)
heroi = Personagem("Heroi", espada)
heroi.atacar()

print(heroi.arma.nome)
```

```
Heroi ataca!
Espada causa 8 de dano
Espada
```

Macete

Leia `self.arma.usar()` da esquerda para a direita, seguindo as "setas": `self` (o personagem) → `.arma` (a espada que ele guarda) → `.usar()` (o método da espada). O personagem não sabe calcular o dano: ele **delega** para a arma. Cada objeto cuida do que é seu.

Outros exemplos de "tem um" no jogo: personagem e inventário, jogo e fase atual, fase e mapa, jogador e placar.

Referência, não cópia

`heroi.arma` e `espada` são o **mesmo objeto** na memória: são dois caminhos que levam ao mesmo lugar. Alterar um altera o outro:

```
class Arma:
    def __init__(self, nome, dano):
```

```
        self.nome = nome
        self.dano = dano


class Personagem:
    def __init__(self, nome, arma):
        self.nome = nome
        self.arma = arma


espada = Arma("Espada", 8)
heroi = Personagem("Heroi", espada)
aliado = Personagem("Aliado", espada)

espada.dano = 12
print(heroi.arma.dano, aliado.arma.dano)
```

```
12 12
```

Atenção

Esse é o ponto preferido das questões de leitura de código. Atribuir um objeto (a um atributo, a outra variável ou a uma lista) **não cria uma cópia**: cria mais um "caminho" para o mesmo objeto. Se dois personagens recebem a mesma espada e o dano dela muda, os dois são afetados.

6. Relação "tem vários"

Quando um objeto precisa guardar uma coleção de objetos, usamos uma **lista como atributo**.

```
class Inimigo:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

    def exibir(self):
        print(f"{self.nome} - vida: {self.vida}")


class Fase:
    def __init__(self, nome):
        self.nome = nome
        self.inimigos = []

    def adicionar_inimigo(self, inimigo):
        self.inimigos.append(inimigo)

    def exibir_inimigos(self):
        print(f"Fase: {self.nome}")
        for inimigo in self.inimigos:
            inimigo.exibir()


fase1 = Fase("Floresta")
fase1.adicionar_inimigo(Inimigo("Goblin", 10))
fase1.adicionar_inimigo(Inimigo("Morcego", 5))
```

```
fase1.adicionar_inimigo(Inimigo("Slime", 8))
fase1.exibir_inimigos()
```

```
Fase: Floresta
Goblin - vida: 10
Morcego - vida: 5
Slime - vida: 8
```

A lista começa **vazia** no construtor, cada inimigo é **guardado** com `append` e a fase pede para **cada** inimigo se exibir. O laço chama `exibir()` sem pensar em qual inimigo é: o que aparece na tela depende dos atributos do objeto que chamou o método.

Atenção

A lista precisa ser criada **dentro do** `__init__` com `self.inimigos = []`. Assim, cada fase tem a sua própria lista. E repare: `adicionar_inimigo` recebe um inimigo como **parâmetro** (usa) e o guarda na lista (tem vários). Uma relação "tem vários" quase sempre é alimentada por um método do tipo "usa".

7. As três relações juntas

O exercício da aula: um personagem **tem um** inventário, o inventário **tem vários** itens e o personagem **usa** o item ao coletá-lo, repassando-o ao inventário.

```
class Item:
    def __init__(self, nome, valor):
        self.nome = nome
        self.valor = valor


class Inventario:
    def __init__(self):
        self.itens = []

    def adicionar_item(self, item):
        self.itens.append(item)

    def exibir_itens(self):
        print("Itens no inventario:")
        for item in self.itens:
            print(f"{item.nome} - valor: {item.valor}")


class Personagem:
    def __init__(self, nome):
        self.nome = nome
        self.inventario = Inventario()

    def coletar(self, item):
        self.inventario.adicionar_item(item)
        print(f"{self.nome} coletou {item.nome}")


heroi = Personagem("Heroi")
heroi.coletar(Item("Moeda", 10))
heroi.coletar(Item("Pocao", 25))
```

```
heroi.inventario.exibir_itens()

print(heroi.inventario.itens[0].nome)
```

```
Heroi coletou Moeda
Heroi coletou Pocao
Itens no inventario:
Moeda - valor: 10
Pocao - valor: 25
Moeda
```

Macete

Para ler uma cadeia como `heroi.inventario.itens[0].nome`, siga as setas uma de cada vez: `heroi` é um Personagem → `.inventario` é o Inventario dele → `.itens` é a lista do inventário → `[0]` é o primeiro Item → `.nome` é o texto "Moeda". Ponto entra em objeto; colchete entra em lista.

Observe que, neste exemplo, o inventário é criado **dentro do** `__init__` do personagem (`self.inventario = Inventario()`), em vez de ser recebido por parâmetro como a arma. As duas formas são "tem um"; a diferença é se o objeto relacionado já existe fora (a espada) ou se nasce junto com o dono (o inventário).

8. Padrões de métodos com "tem vários"

Dentro da classe que guarda a lista, os padrões de lista continuam valendo, só que percorrendo objetos:

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco


class Carrinho:
    def __init__(self):
        self.produtos = []

    def adicionar(self, produto):
        self.produtos.append(produto)

    def total(self):
        soma = 0
        for p in self.produtos:
            soma += p.preco
        return soma

    def mais_caro(self):
        if len(self.produtos) == 0:
            return None
        maior = self.produtos[0]
        for p in self.produtos:
            if p.preco > maior.preco:
                maior = p
        return maior
```

```python
    def buscar(self, nome):
        for p in self.produtos:
            if p.nome == nome:
                return p
        return None


c = Carrinho()
c.adicionar(Produto("Mouse", 120))
c.adicionar(Produto("Monitor", 900))
c.adicionar(Produto("Cabo", 20))

print(c.total())
print(c.mais_caro().nome)
print(c.buscar("Cabo").preco)
print(c.buscar("Teclado"))
```

```
1040
Monitor
20
None
```

Macete

Três métodos que aparecem em quase toda questão de "tem vários": **total** (acumulador), **maior/menor** (guardando o objeto) e **buscar** (`return` do objeto dentro do laço, `return None` depois do laço). Devolver o **objeto** em vez de um número permite usar qualquer atributo dele depois, como em `c.mais_caro().nome`.

9. Erros mais comuns

Atenção

- **Criar a lista fora do `__init__`:** a lista precisa ser `self.itens = []` dentro do construtor.
- **Guardar o nome em vez do objeto:** `self.itens.append(item.nome)` perde o acesso ao valor e aos métodos do item. Guarde o objeto inteiro.
- **Achar que a atribuição copia:** `self.arma = arma` não cria uma arma nova.
- **Esquecer os parênteses ao criar:** `self.inventario = Inventario` guarda a classe, e não um objeto. O correto é `Inventario()`.
- **Acessar atributo de um objeto que é None:** se `buscar` não encontra e devolve `None`, fazer `.nome` logo em seguida dá `AttributeError`. Teste antes com `if resultado != None`.
- **Fazer o trabalho do outro objeto:** o personagem calculando o dano da arma com `self.arma.dano * 2` em vez de pedir `self.arma.usar()`. Funciona, mas quebra a ideia de que cada objeto cuida dos seus dados.

10. Questões de exemplo resolvidas

Questão 1 — Modelagem: biblioteca (exercício de fixação 1)

Um sistema tem livros, leitores, empréstimos e funcionários. Identifique as classes, com pelo menos 3 atributos e 2 métodos cada, e as relações.
Uma resposta possível.

Classe	Atributos	Métodos
Livro	título, autor, disponível	`exibir()`, `marcar_emprestado()`
Leitor	nome, matrícula, empréstimos	`pegar(livro)`, `devolver(livro)`
Emprestimo	livro, leitor, data, devolvido	`finalizar()`, `exibir()`
Funcionario	nome, cargo, matrícula	`registrar_emprestimo()`, `cadastrar_livro()`

Relações: um empréstimo **tem um** livro e **tem um** leitor; um leitor **tem vários** empréstimos; o funcionário **usa** o livro e o leitor ao registrar um empréstimo.

Questão 2 — Consulta médica (exercício de fixação 2)

Crie as classes `Paciente` (nome, idade) e `Consulta` (especialidade, valor). Implemente `agendar(consulta)` para que o paciente registre uma consulta e exiba uma confirmação.
Raciocínio. "Agendar" recebe a consulta: relação **usa**. "Registrar" significa guardar: o paciente **tem várias** consultas, então precisa de uma lista.

```
class Consulta:
    def __init__(self, especialidade, valor):
        self.especialidade = especialidade
        self.valor = valor


class Paciente:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
        self.consultas = []

    def agendar(self, consulta):
        self.consultas.append(consulta)
        print(f"{self.nome}: consulta de {consulta.especialidade} "
              f"agendada (R$ {consulta.valor:.2f})")

    def total_gasto(self):
        soma = 0
        for c in self.consultas:
            soma += c.valor
        return soma


p = Paciente("Joana", 34)
p.agendar(Consulta("Cardiologia", 250))
p.agendar(Consulta("Dermatologia", 180))
print(len(p.consultas), p.total_gasto())
```

```
Joana: consulta de Cardiologia agendada (R$ 250.00)
Joana: consulta de Dermatologia agendada (R$ 180.00)
```

```
2 430
```

Questão 3 — Pedido com as três relações (exercício de fixação 3)

Crie as classes `Cliente`, `Produto` e `Pedido`. O pedido tem um cliente, guarda uma lista de produtos e possui `adicionar_produto(produto)` e `exibir_resumo()`.

Raciocínio. Pedido **tem um** cliente (atributo recebido no construtor), **tem vários** produtos (lista) e **usa** cada produto ao adicioná-lo.

```
class Cliente:
    def __init__(self, nome):
        self.nome = nome


class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco


class Pedido:
    def __init__(self, cliente):
        self.cliente = cliente
        self.produtos = []

    def adicionar_produto(self, produto):
        self.produtos.append(produto)

    def exibir_resumo(self):
        print("Cliente:", self.cliente.nome)
        total = 0
        for p in self.produtos:
            print(f"  {p.nome}: R$ {p.preco:.2f}")
            total += p.preco
        print(f"Total: R$ {total:.2f}")


ana = Cliente("Ana")
mouse = Produto("Mouse", 120)
pedido = Pedido(ana)
pedido.adicionar_produto(mouse)
pedido.adicionar_produto(Produto("Cabo", 20))

mouse.preco = 100
pedido.exibir_resumo()
```

```
Cliente: Ana
  Mouse: R$ 100.00
  Cabo: R$ 20.00
Total: R$ 120.00
```

O preço do mouse foi alterado **depois** de adicionado, e o resumo mostra 100: a lista do pedido guarda uma referência ao mesmo objeto.

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```
class Arma:
    def __init__(self, nome, dano):
        self.nome = nome
        self.dano = dano


class Personagem:
    def __init__(self, nome, arma):
        self.nome = nome
        self.arma = arma

    def trocar_arma(self, nova):
        antiga = self.arma
        self.arma = nova
        return antiga


faca = Arma("Faca", 3)
arco = Arma("Arco", 6)

a = Personagem("Ana", faca)
b = Personagem("Bia", faca)

faca.dano = 4
largada = a.trocar_arma(arco)
largada.dano += 1

print(a.arma.nome, a.arma.dano)
print(b.arma.nome, b.arma.dano)
print(largada.nome, faca.dano)
```

Raciocínio. Ana e Bia começam com a **mesma** faca. O dano da faca vira 4. Ana troca para o arco, e o método devolve a faca antiga: `largada` é a faca. Somar 1 ao dano de `largada` muda a faca, que ainda está com a Bia.

```
Arco 6
Faca 5
Faca 5
```

11. Cola rápida

Quero...	Escrevo...
usar outro objeto (usa)	`def coletar(self, item):` e acessar `item.valor`
guardar um objeto recebido (tem um)	`self.arma = arma`
criar o objeto junto com o dono (tem um)	`self.inventario = Inventario()`
guardar vários (tem vários)	`self.itens = []` no `__init__`
adicionar à coleção	`self.itens.append(item)`
pedir para o objeto guardado agir	`self.arma.usar()`
percorrer a coleção	`for item in self.itens: item.exibir()`
buscar na coleção	`return` do objeto no laço; `return None` após o laço
seguir uma cadeia	`heroi.inventario.itens[0].nome`